

3-Mark Questions & Answers — Data Structures Module 1

Topic 1: Concept of ADT

Q1. Explain the concept of ADT with a suitable example.

An **Abstract Data Type (ADT)** is a high-level specification of a set of data items and the operations performed on them, completely independent of how the data is stored in memory or how operations are implemented.

- **Example: List ADT**
 - **Data Domain:** A sequence of elements $[e_1, e_2, \dots, e_n]$.
 - **Operations:**
 - `insert(pos, element)` : Insert element at specified position.
 - `delete(pos)` : Remove element from specified position.
 - `get(pos)` : Return element at position.
 - **Implementation Independence:** A List ADT can be implemented using either a contiguous Array or a dynamic Linked List without changing how the user interacts with the ADT interface.
-

Q2. What are the advantages of using ADTs in programming?

1. **Modularity & Reusability:** Software is broken into independent modules. An ADT implementation can be modified or optimized without altering client code.
 2. **Information Hiding & Abstraction:** Users interact only with the clean public interface, shielding internal implementation details and data representations.
 3. **Easier Debugging & Maintenance:** Bugs inside an ADT implementation are localized and fixed independently without affecting external application logic.
-

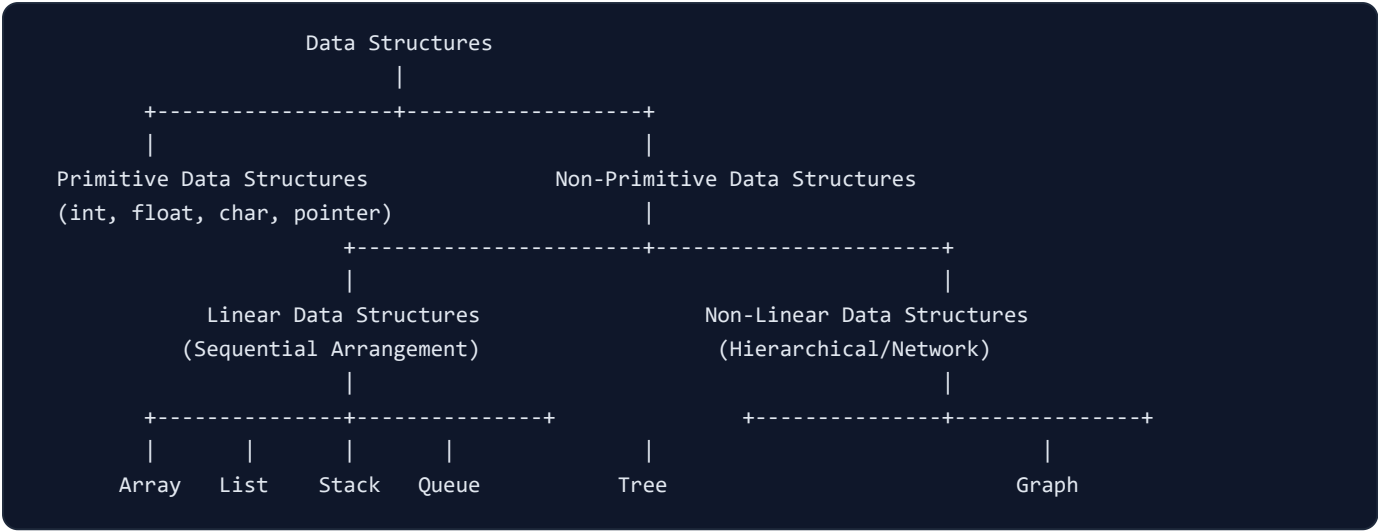
Q3. Explain the relationship between ADT and encapsulation.

- **Encapsulation** is the object-oriented programming mechanism of wrapping data and methods into a single unit (class/structure) and restricting direct access to private data.
 - **Relationship:** ADT defines the **conceptual specification** (what operations exist), whereas **Encapsulation** provides the **language-level implementation mechanism** (using `private` fields and `public` methods in C++/Java or opaque structure pointers in C) to enforce the boundaries defined by the ADT.
-

Topic 2: Types of Data Structures

Q4. Classify data structures with a neat diagram.

Data structures are categorized into Primitive and Non-Primitive, with Non-Primitive further split into Linear and Non-Linear structures:



Q5. Explain primitive vs non-primitive data structures with examples.

Property	Primitive Data Structures	Non-Primitive Data Structures
Definition	Basic data types directly supported by hardware/CPU.	Complex structures formed by combining primitive data types.
Memory Size	Fixed size defined by language compiler.	Variable size dependent on element count.
Focus	Storing single basic values.	Organizing and managing groups of related data elements.
Examples	int , float , char , double , void* .	Arrays, Linked Lists, Stacks, Queues, Trees, Graphs.

Q6. Compare linear and non-linear data structures (any three points).

Feature	Linear Data Structures	Non-Linear Data Structures
Arrangement	Elements are arranged sequentially in single level.	Elements are arranged hierarchically in multiple levels.
Traversal	Single run traversal can visit all elements sequentially.	Requires complex tree/graph traversal (DFS/BFS).
Memory Utilization	Can lead to inefficient memory use (e.g., linear queues).	Efficient memory usage according to non-linear relations.
Examples	Array, Linked List, Stack, Queue.	Tree, Binary Search Tree, Graph.

Topic 3: Operations on Data Structures

Q7. Explain insertion and deletion operations with examples.

- **Insertion Operation:** The process of adding a new data element into an existing data structure.

- *Example:* Adding **50** to the end of an array `arr[4] = {10, 20, 30, 40}` produces `arr[5] = {10, 20, 30, 40, 50}` (provided capacity exists).
- **Deletion Operation:** The process of removing an existing data element from a data structure and shifting remaining elements if necessary.
- *Example:* Deleting index **1** (**20**) from `arr = {10, 20, 30}` shifts elements left to produce `arr = {10, 30}`.

Q8. Explain sorting and merging operations on data structures.

- **Sorting Operation:** Arranging elements of a data structure in a specific numerical or alphabetical order (Ascending or Descending).
- *Example:* Sorting `{34, 12, 5, 89}` in ascending order produces `{5, 12, 34, 89}` (using Bubble Sort or Quick Sort).
- **Merging Operation:** Combining elements of two sorted data structures into a single sorted data structure.
- *Example:* Merging sorted Array A `{2, 5, 8}` and Array B `{1, 4, 9}` produces sorted Array C `{1, 2, 4, 5, 8, 9}` in **O(m+n)** time.

Q9. Differentiate between traversal and searching operations.

Feature	Traversal Operation	Searching Operation
Objective	Visit every single element in the structure once.	Locate a specific target value.
Execution	Always inspects 100% of elements.	Stops immediately once target element is found.
Output	Performs an action on all elements (e.g., print all).	Returns index/location of target or -1 (not found).

Topic 4: Arrays & Multi-Dimensional Arrays

Q10. Explain row-major and column-major order representation of a 2-D array.

A 2-D array is logically a matrix but stored in 1-D contiguous physical memory:

1. **Row-Major Order (RMO):** Elements are stored row by row. All elements of Row 0 are stored first, followed by Row 1, Row 2, etc. (Default in C/C++/Java).
2. **Column-Major Order (CMO):** Elements are stored column by column. All elements of Column 0 are stored first, followed by Column 1, Column 2, etc. (Default in Fortran/MATLAB).

Q11. Derive the address calculation formula for an element in a 2-D array (row-major).

For a 2-D array **A[m][n]** stored in Row-Major Order with Base Address **B** and element size **w**:

- To reach element **A[i][j]**, we must skip **i** complete rows (each row contains **n** columns).
- Within row **i**, we skip **j** column elements.
- Total elements skipped = **(i × n) + j**.
- **Formula:**

Equation:

$$\text{Address of } A[i][j] = B + w \times [(i - LBR) \times n + (j - LBC)]$$

(Assuming $LBR = LBC = 0$, $\text{Address} = B + w \times (i \times n + j)$).

Q12. Write a short note on sparse matrices.

A **Sparse Matrix** is a matrix in which the majority of elements are zero.

- **Problem with Standard Representation:** Storing an $n \times n$ sparse matrix with 90% zeros in a 2-D array wastes significant memory.

- **Solution (Triplet Representation):** Store only non-zero elements using a 3-column table:

1. **Row Index**

2. **Column Index**

3. **Non-Zero Value**

- **Benefit:** Reduces memory complexity from $O(m \times n)$ down to $O(k)$ where k is the number of non-zero elements.

Topic 5: String Manipulation

Q13. Explain any three string manipulation functions with examples.

1. `strlen(s)` : Calculates length excluding null character.

- Example: `strlen("HELLO")` returns `5` .

2. `strcpy(dest, src)` : Copies characters from `src` to `dest` .

- Example: `strcpy(str, "DATA")` sets `str` to `"DATA"` .

3. `strcat(dest, src)` : Appends `src` to end of `dest` .

- Example: `strcat(a, b)` where `a="B.Tech "` and `b="DS"` results in `a="B.Tech DS"` .

Q14. Write a note on pattern matching in strings.

Pattern Matching is the process of checking whether a specific substring (called **Pattern P** of length m) exists inside a larger text string (called **Text T** of length n).

- **Naive Algorithm:** Compares **P** against every sliding window of **T**. Time complexity: $O(m \times n)$.

- **Advanced Algorithms:** KMP (Knuth-Morris-Pratt), Rabin-Karp, and Boyer-Moore algorithms preprocess the pattern to achieve linear $O(n)$ matching.

Q15. Explain the concept of string representation as an array of characters.

In languages like C, strings are represented as 1-D arrays of `char` primitive elements.

- **Storage:** Sequential contiguous bytes ending with null byte `'\0'` .

- **Declaration:** `char str[10] = "DS";`

- **Memory Map:**

- `str[0] = 'D'` (ASCII 68)

- `str[1] = 'S'` (ASCII 83)

- `str[2] = '\0'` (ASCII 0)

- **Benefit:** Allows index-based character modification (`str[0] = 'C'`).

Topic 6: Self-Referential Structures with Pointers

Q16. Explain self-referential structures with a suitable example and diagram.

A **Self-Referential Structure** contains a pointer variable pointing to another structure of the same type.

```
struct Node {
    int data;
    struct Node *next;
};
```

Diagram:

```
+-----+-----+      +-----+-----+
| Data | Next | ---> | Data | Next | ---> NULL
|  10  | *---|-----> |  20  | NULL |
+-----+-----+      +-----+-----+
```

Q17. Explain how self-referential structures are used to implement linked lists.

Dynamic linked lists cannot use static contiguous arrays. Instead:

1. Each element is created dynamically as a `struct Node` using `malloc(sizeof(struct Node))`.
2. The `data` field holds the payload.
3. The self-referential pointer `next` stores the memory address returned by `malloc()` for the subsequent node.
4. Setting `last->next = NULL` marks the list termination.

Q18. Differentiate between arrays and self-referential (pointer-based) structures.

Feature	Arrays	Self-Referential Structures
Memory Allocation	Contiguous memory allocated at compile time.	Non-contiguous memory allocated dynamically at runtime.
Insertion / Deletion	Slow O(n) due to element shifting.	Fast O(1) by updating pointer links.
Memory Overhead	Zero pointer overhead.	Extra pointer size (4 or 8 bytes per node).